

NOTA DE CLASE 1:

Introducción.

Software: Instrucciones (programas de cómputo), procedimientos, reglas, documentación y datos asociados que forman parte de las operaciones de un sistema de computación (IEEE).

Características del software: elemento lógico, se desarrolla, no se fabrica, no se desgasta.

Software libre: El término software libre (o programas libres) se refiere a libertad, tal como fue concebido por Richard Stallman en su definición. En concreto, se refiere a cuatro libertades:

1. Libertad para ejecutar el programa en cualquier sitio, con cualquier propósito y para siempre.
2. Libertad para estudiarlo y adaptarlo a nuestras necesidades. Esto exige el acceso al código fuente.
3. Libertad de redistribución, de modo que se nos permita colaborar con vecinos y amigos.
4. Libertad para mejorar el programa y publicar las mejoras. También exige el código fuente.

Clasificación del software: De sistemas; De aplicación; Científico y de ingeniería; Integrado; De línea de productos; Aplicaciones Web/Móviles; De inteligencia artificial.

Software – Retos: A medida que los usuarios adoptan nuevas tecnologías, parte del trabajo consiste en integrar los sistemas tradicionales con los nuevos para asegurar un contexto útil. Debemos estar conscientes de que, al integrar tecnologías, se ven afectados todos los tipos de usuarios y sistemas.

¿Qué es la Ingeniería de Software?: Disciplina de la ingeniería (Hace que las cosas funcionen; se aplican teorías, métodos y herramientas) que comprende todos los aspectos de la producción de software desde las etapas iniciales de la especificación del sistema, incluyendo la evolución de éste, hasta que se comienza a ejecutar. No sólo comprende los procesos técnicos del desarrollo de software, sino también se realizan actividades como la gestión de proyectos y el desarrollo de herramientas, métodos y teorías de apoyo a la producción de software.

La IEEE define a la Ingeniería de Software como:

1. El uso de métodos sistemáticos, disciplinados y cuantificables para el desarrollo, operación y mantenimiento de software.
2. El estudio de técnicas relacionadas con 1.

La cuantificación rigurosa de recursos, procesos y productos es una precondition para optimizar productividad y calidad. La “metrificación” y el control estadístico de procesos son claves en Ingeniería de Software.

Un Ingeniero de Software debe cumplir contratos en tiempo y costos como es normal en obras de Ingeniería. Ello presupone la capacidad de medir, estimar, planificar y administrar proyectos.

La Ingeniería de Software se ocupa de todo el ciclo de vida de un producto, desde su etapa inicial de planificación y análisis de requerimientos hasta la estrategia para determinar cuándo y cómo debe ser retirado de servicio.

Participantes en el desarrollo de software:

- Gerentes ejecutivos (dueños del producto).
- Gerentes del proyecto (líder de equipo).
- Profesionales especializados.
- Clientes.
- Usuarios finales.

Características de un Ingeniero de Software: El Ingeniero debe dominar los aspectos técnicos, aprender habilidades requeridas para entender el problema, diseñar solución, desarrollarla, etc. Pero, además, los aspectos humanos son lo que lo harán un ingeniero efectivo. Tener un sentido de responsabilidad individual, aguda conciencia de las necesidades del equipo, atención al detalle entre otros.

Responsabilidad profesional y ética: La Ingeniería de Software se desarrolla en un marco económico, social y legal. Los IS deben aceptar responsabilidades más amplias que las responsabilidades técnicas. No debe utilizar su capacidad y habilidades de forma deshonesto o de forma que deshonre la profesión.

- Confidencialidad: Respetar la confidencialidad de sus empleados y clientes.
- Competencia: No falsificar el nivel de competencia y aceptar responsabilidades fuera de su capacidad.
- Derechos de la propiedad intelectual: Conocer la leyes vigentes sobre las patentes y copyright.
- Uso inapropiado de las computadoras: No debe utilizar sus habilidades técnicas para utilizar de forma inapropiada otras computadoras.

Existen diferentes organizaciones, como ACM o IEEE, que sugieren diferentes códigos de ética a respetar.

Técnicas de comunicación: Al iniciar un proyecto, la primera actividad es saber lo que el usuario quiere, cómo lo quiere, cuándo y por qué. La comunicación es la base para la obtención de las necesidades del cliente. Es la principal fuente de error. Al hablar de necesidades, en términos más técnicos, estamos hablando de requerimientos.

Requerimientos: Un Requerimiento (o requisito) es una característica del sistema o una descripción de algo que el sistema es capaz de hacer con el objeto de satisfacer el propósito del sistema.

Definición IEEE-Std-610:

- Condición o capacidad que necesita el usuario para resolver un problema o alcanzar un objetivo.
- Condición o capacidad que debe satisfacer o poseer un sistema o una componente de un sistema para satisfacer un contrato, un estándar, una especificación u otro documento formalmente impuesto.
- Representación documentada de una condición o capacidad como en 1 o 2.

Fuentes de requerimientos:

- Documentación.
- Stakeholders: El término stakeholder se utiliza para referirse a cualquier persona o grupo que se verá afectado por el sistema, directa o indirectamente. Entre los stakeholders, se encuentran: usuarios finales, ingenieros, gerentes, expertos del dominio.
- Especificaciones de sistemas similares.

Puntos de vista: Existen tres tipos genéricos de puntos de vista:

- Punto de vista de los interactuadores: Representan a las personas u otros sistemas que interactúan, directamente, con el sistema. Pueden influir en los requerimientos del sistema de algún modo.
- Punto de vista indirecto: Representan a los stakeholders que no utilizan el sistema ellos mismos, pero que influyen en los requerimientos de algún modo.
- Punto de vista del dominio: Representan las características y restricciones del dominio que influyen en los requerimientos del sistema.

Elicitación de requerimientos: Es el proceso de adquirir (“eliciting”) [sonsacar] todo el conocimiento relevante necesario para producir un modelo de los requerimientos de un dominio de problema.

Objetivos:

- Conocer el dominio del problema para poder comunicarse con clientes y usuarios y entender sus necesidades.
- Conocer el sistema actual (manual o informatizado).
- Identificar las necesidades, tanto explícitas como implícitas, de clientes y usuarios y sus expectativas sobre el sistema a desarrollar.

La elicitación de requisitos es una actividad, principalmente, de carácter social, mucho más que tecnológico. Los problemas que se plantean son, por lo tanto, de naturaleza psicológica y social, más que técnicos.

Problemas de comunicación:

- Dificultad para expresar, claramente, las necesidades.
- No ser conscientes de sus propias necesidades.
- No entender cómo la tecnología puede ayudar.
- Miedo a parecer incompetentes por ignorancia tecnológica.
- No tomar decisiones por no poder prever las consecuencias, no entender las alternativas
- no tener una visión global.
- Cultura y vocabulario diferentes.
- Intereses distintos en el sistema a desarrollar.
- Medios de comunicación inadecuados (diagramas que no entienden los clientes y usuarios).
- Conflictos personales o políticos.

Limitaciones cognitivas:

- No conocer el dominio del problema.
- Hacer suposiciones sobre el dominio del problema.
- Hacer suposiciones sobre aspectos tecnológicos.
- Hacer simplificaciones excesivas.

Conducta humana:

- Conflictos y ambigüedades en los roles de los participantes.
- Pasividad de clientes, usuarios o ingenieros de requisitos.
- Temor a que el nuevo sistema lo deje sin trabajo.

Técnicos:

- Complejidad del dominio del problema.
- Complejidad de los requisitos.
- Múltiples fuentes de requisitos.
- Fuentes de información poco claras.

TÉCNICAS DE ELICITACIÓN:

Métodos discretos:

1. Muestreo de la documentación, los formularios y los datos existentes.
2. Investigación y visitas al lugar.
3. Observación del ambiente de trabajo.

Métodos interactivos:

1. Cuestionarios.
2. Entrevistas.
3. Planeación Conjunta de Requerimientos (JRP o JAD).
4. Lluvia de Ideas - Brainstorming.

MÉTODOS INTERACTIVOS: Hay métodos interactivos que pueden usarse para obtener los requerimientos de los miembros de la organización. Aunque son distintos en su implementación, estos métodos tienen muchas cosas en común. La base es hablar con las personas en la organización y escuchar para comprender. Cada uno cuenta con su propio proceso establecido.

Cuestionarios: Documento que permite al analista recabar información y opiniones de los encuestados; recolectar hechos de un gran número de personas; detectar un sentimiento generalizado; detectar problemas entre usuarios; cuantificar respuestas.

Ventajas:

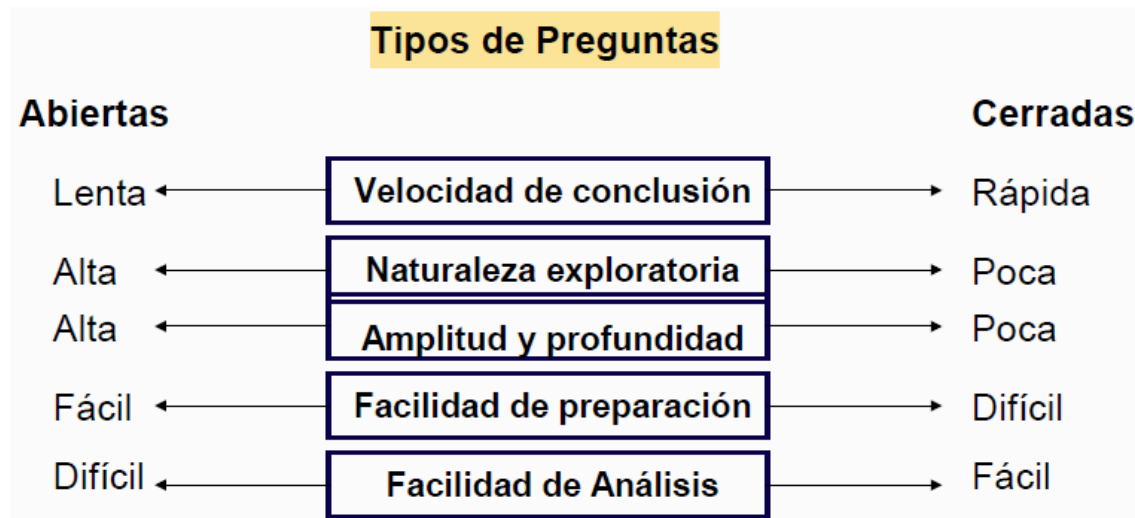
- Respuesta rápida.
- Económicos.
- Anónimos.
- Estructurados de fácil análisis.

Desventajas:

- Número bajo de respuestas.
- No responde a todas las preguntas.
- Preguntas rígidas.
- No se puede realizar el análisis corporal.
- No se pueden aclarar respuestas incompletas.
- Dificiles de preparar.

Tipos de preguntas:

- Abiertas: Son las que dejan abiertas todas las posibles opciones de respuesta.
- Cerradas: Limitan o cierran las opciones de respuestas disponibles.



Tipo de información obtenida:

- Actitud: Lo que las personas dicen que quieren.
- Creencias: Lo que las personas creen que es verdad.
- Comportamiento: Lo que, realmente, hacen.
- Características: De las personas o cosas.

Cuándo usar cuestionarios:

- Las personas están dispersas geográficamente (diferentes oficinas o ciudades).
- Muchas personas involucradas (clientes o usuarios).
- Queremos obtener opiniones generales.
- Queremos identificar problemas generales.

Entrevistas: Técnica de exploración mediante la cual el analista de sistemas recolecta información de las personas a través de la interacción cara a cara. Es una conversación con un propósito específico, que se basa en un formato de preguntas y respuestas en general. Conocer opiniones y sentimientos del entrevistado.

Tipo de información obtenida:

- Opiniones.
- Objetivos.
- Procedimientos informales.
- Sentimientos.

Ventajas:

- El entrevistado se siente incluido en el proyecto.
- Es posible obtener una retroalimentación del encuestado.
- Es posible adaptar las preguntas de acuerdo al entrevistado.
- Información no verbal observando las acciones y expresiones del entrevistado.

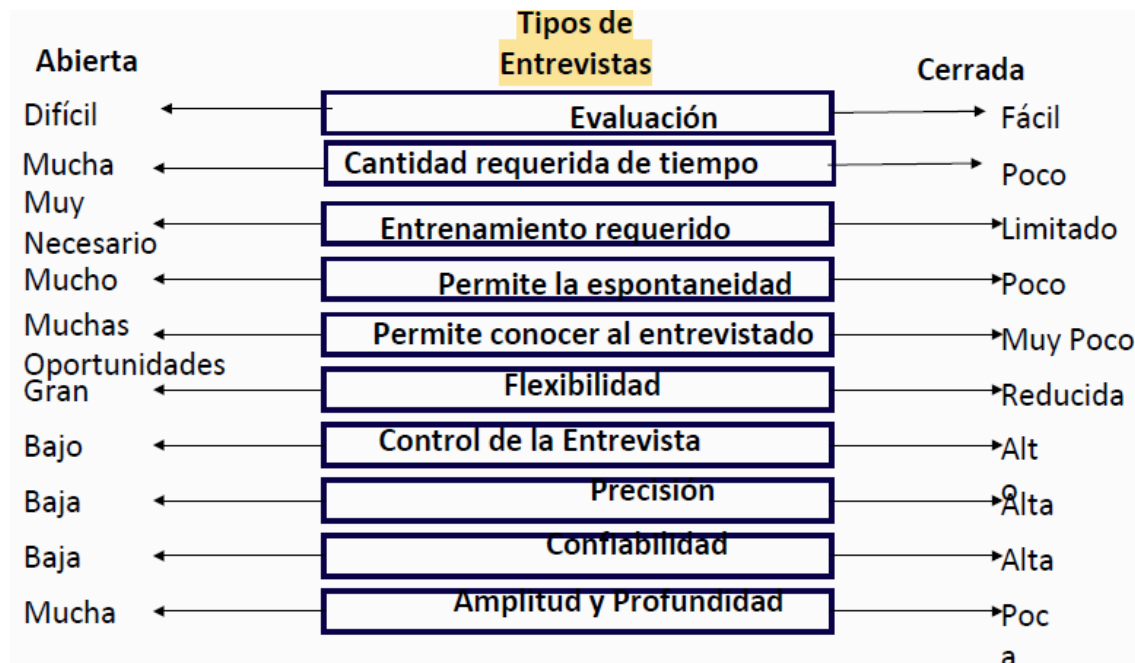
Desventajas:

- Costosas.
- Tiempo y recursos humanos.

- Las entrevistas dependen, en gran parte, de las habilidades del entrevistador.
- No aplicable a distancia.

Tipos de entrevistas:

- Estructuradas (Cerradas):
 - El encuestador tiene un conjunto específico de preguntas para hacérselas al entrevistado.
 - Se dirige al usuario sobre un requerimiento puntual.
 - No permite adquirir un amplio conocimiento del dominio.
- No estructuradas (Abiertas):
 - El encuestador lleva a un tema en general.
 - Sin preparación de preguntas específicas.
 - Iniciar con preguntas que no dependen del contexto, para conocer el problema, la gente involucrada, etc.



Tipos de preguntas:

- Abiertas: Permite al encuestado responder de cualquier manera. Ventajas: revelan nueva línea de preguntas; hacen más interesante la entrevista; permiten espontaneidad. Desventajas: pueden dar muchos detalles irrelevantes; se puede perder el control de la entrevista; parece que el entrevistador no tiene los objetivos claros.
- Cerradas: Las respuestas son directas, cortas o de selección específica. Ventajas: ahorran tiempo; se mantiene más fácil el control de la entrevista; se consiguen datos relevantes. Desventajas: pueden aburrir al encuestado; no se obtienen detalles.
- Sondeo: Permite obtener más detalle sobre un tema puntual.

Organización de una entrevista:

- Piramidal (Inductivo): Preguntas cerradas a Preguntas abiertas.
- Embudo (Deductivo): Preguntas abiertas a Preguntas cerradas.
- Diamante (Combinación de las anteriores): Preguntas cerradas a Preguntas abiertas a Preguntas cerradas.

Preparación previa (Kendall):

- Leer los antecedentes: Poner atención en el lenguaje. Buscar un vocabulario en común. Imprescindible para poder entender al entrevistado.
- Establecer los objetivos de la entrevista: Usando los antecedentes. Los directivos suelen proporcionar una visión general, mientras que los futuros usuarios una más detallada.
- Seleccionar los entrevistados: Se debe minimizar el número de entrevistas. Los entrevistados deben conocer, con antelación, el objetivo de la entrevista y las preguntas que se le van a hacer.
- Planificación de la entrevista y preparación del entrevistado: Establecer fecha, hora, lugar y duración de cada entrevista de acuerdo con el entrevistado.
- Selección del tipo de preguntas a usar y su estructura.

Planeación Conjunta de Requerimientos (JRP): Proceso mediante el cual se conducen reuniones de grupos altamente estructurados con el propósito de analizar problemas y definir requerimientos.

Características:

- Requiere de extenso entrenamiento.
- Reduce el tiempo de exploración de requisitos.
- Amplia participación de los integrantes.
- Se trabaja sobre lo que se va generando.

Ventajas:

- Ahorro de tiempo.
- Usuarios involucrados.
- Desarrollos creativos.

Desventajas:

- Es difícil organizar los horarios de los involucrados.
- Es complejo encontrar un grupo de participantes integrados y organizados.

Lluvia de Ideas (Brainstorming): Técnica para generar ideas al alentar a los participantes para que ofrezcan tantas ideas como sea posible en un corto tiempo sin ningún análisis hasta que se hayan agotado las ideas. Se promueve el desarrollo de ideas creativas para obtener soluciones. Se realizan reuniones del equipo involucrado en la resolución del problema, conducidas por un director.

- Incluye una serie de fases de aplicación: descubrir hechos, producir ideas, descubrir ideas.
- Clave para resolver la falta de consenso entre usuarios.
- Es útil combinarlo con la toma de decisiones.
- Ayuda a mantener el dominio del problema.
- Encara la dificultad del usuario para transmitir.
- Ayuda a entender al usuario y al analista.

MÉTODOS DISCRETOS: Los métodos discretos son menos perturbadores que otras formas de averiguar los requerimientos. Se consideran insuficientes para recopilar información cuando se utilizan por sí solos, por lo que deben utilizarse junto con uno o varios de los métodos. Utilizar diferentes métodos para acercarse a la organización es una práctica inteligente mediante la cual podrá formarse un panorama más completo de los requerimientos

Muestreo de la documentación, los formularios y los datos existentes:

Recolección de hechos a partir de la documentación existente. ¿Qué tipo de documentos pueden enseñar algo acerca del sistema?:

- Organigrama (identificar el propietario, usuarios claves).
- Memos, notas internas, minutas, registros contables.
- Solicitudes de proyectos de sistemas de información anteriores.
- Documentos que describen la funcionalidad del negocio que está siendo analizada.
- Documentación de sistemas anteriores.

Investigación y visitas al lugar:

- Investigar el dominio.
- Patrones de soluciones (misma problema en otra organización).
- Revistas especializadas.
- Buscar problemas similares en internet.
- Consultar otras organizaciones.

Observación del ambiente de trabajo: El analista se convierte en observador de las personas y actividades con el objeto de aprender acerca del sistema.

Lineamientos de la observación:

- Determinar quién y cuándo será observado.
- Obtener el permiso de la persona y explicar el porqué será observado.
- Mantener bajo perfil.
- Tomar nota de lo observado.
- Revisar las notas con la persona apropiada.
- No interrumpir a la persona en su trabajo.

Ventajas:

- Datos confiables.
- El analista puede ver, exactamente, lo que se hace (tareas difíciles de explicar con palabras).
- Análisis de disposiciones físicas, tránsito, iluminación, ruido.
- Económica en comparación con otras técnicas.

Desventajas:

- La gente se siente incómoda siendo observada.
- Algunas actividades del sistema pueden ser realizadas en horarios incómodos.
- Las tareas están sujetas a interrupciones.
- Tener en cuenta que la persona observada puede estar realizando las tareas de la forma “correcta” y no como lo hace habitualmente.

ESPECIFICACIÓN DE REQUERIMIENTOS:

Objetivos:

- Permitir que los desarrolladores expliquen cómo han entendido lo que el cliente pretende del sistema.
- Indicar a los diseñadores qué funcionalidad y características va a tener el sistema resultante.
- Indicar al equipo de pruebas qué demostraciones llevar a cabo para convencer al cliente de que el sistema que se le entrega es lo que había pedido.

Propiedades de los requerimientos:

- Necesario: Su omisión provoca una deficiencia.
- Conciso: Fácil de leer y entender.
- Completo: No necesita ampliarse.
- Consistente: No contradictorio con otro.
- No ambiguo: Tiene una sola implementación.
- Verificable: Puede testearse a través de inspecciones, pruebas, etc.

Aspectos básicos de una especificación de requerimientos:

- Funcionalidad: ¿Qué debe hacer el software?
- Interfaces externas: ¿Cómo interactuará el software con el medio externo (gente, hardware, otro software)?
- Rendimiento: Velocidad, disponibilidad, tiempo de respuesta, etc.
- Atributos: Portabilidad, seguridad, mantenibilidad, eficiencia.
- Restricciones de Diseño: Estándares requeridos, lenguaje, límite de recursos, etc.

Técnicas de especificación de requerimientos:

- Estáticas: Se describe el sistema a través de las entidades u objetos, sus atributos y sus relaciones con otros. No describe cómo las relaciones cambian con el tiempo. Cuando el tiempo no es un factor mayor en la operación del sistema, es una descripción útil y adecuada. Ejemplos: Referencia indirecta, Relaciones de recurrencia, Definición axiomática, Expresiones regulares, Abstracciones de datos, entre otras.
- Dinámicas: Se considera un sistema en función de los cambios que ocurren a lo largo del tiempo. Se considera que el sistema está en un estado particular hasta que un estímulo lo obliga a cambiar su estado. Ejemplos: Tablas de decisión, Diagramas de transición de estados, Tablas de transición de estados, Diagramas de persianas, Diagramas de transición extendidos, Redes de Petri, entre otras.

NOTA DE CLASE 2:

Historias de Usuario (HU).

Definición: Una historia de usuario es una descripción corta y simple de un requerimiento de un sistema, que se escribe en lenguaje común del usuario y desde su perspectiva. Son utilizadas en las metodologías de desarrollo ágiles (Ejemplo: XP, SCRUM) para la especificación de requerimientos, acompañadas de las discusiones con los usuarios y las pruebas de validación.

Conceptos:

- Debe ser limitada, ésta debería poder escribirse sobre una nota adhesiva pequeña.
- Son una forma rápida de administrar los requisitos de los usuarios sin tener que elaborar gran cantidad de documentos formales y sin requerir de mucho tiempo para administrarlos.
- Permiten responder, rápidamente, a los requisitos cambiantes.
- Al momento de implementar las historias, los desarrolladores deben tener la posibilidad de discutir las con los clientes.
- Generalmente, se espera que la estimación de tiempo de cada historia de usuario se sitúe entre unas 10 horas y un par de semanas.
- Estimaciones mayores a dos semanas son indicativo de que la historia es muy compleja y debe ser dividida en varias historias.

Forma de redactarlas: Si bien el estilo puede ser libre, la historia de usuario debe responder a tres preguntas: ¿Quién se beneficia? ¿Qué se quiere? ¿Cuál es el beneficio?.

Características:

- **Independientes unas de otras:** De ser necesario, combinar las historias dependientes o buscar otra forma de dividir las historias de manera que resulten independientes.
- **Negociables:** La historia en sí misma no es lo suficientemente explícita como para considerarse un contrato. La discusión con los usuarios debe permitir esclarecer su alcance y éste debe dejarse explícito bajo la forma de pruebas de validación.
- **Valoradas por los clientes o usuarios:** Los intereses de los clientes y de los usuarios no siempre coinciden, pero, en todo caso, debe ser importante para alguno de ellos más que para el desarrollador.
- **Estimables:** Un resultado de la discusión de una historia de usuario es la estimación del tiempo que tomará completarla. Esto permite estimar el tiempo total del proyecto.
- **Pequeñas:** Las historias muy largas son difíciles de estimar e imponen restricciones sobre la planificación de un desarrollo iterativo. Generalmente, se recomienda la consolidación de historias muy cortas en una sola historia.
- **Verificables:** Las historias de usuario cubren requerimientos funcionales, por lo que, generalmente, son verificables. Cuando sea posible, la verificación debe automatizarse, de manera que pueda ser verificada en cada entrega del proyecto.

Criterios de aceptación:

- Un criterio de aceptación es el criterio por el cual se define si una historia de usuario fue desarrollada según la expectativa del Product Manager/Owner (como representante de los criterios del cliente) y si se puede dar como hecha.
- Deben ser definidos durante la etapa inicial antes de la codificación, acompañan a la historia de usuario, porque complementan la historia de usuario y ayudan al equipo de desarrollo a entender mejor cómo se espera que el producto se comporte.

- Los criterios de aceptación son utilizados para expresar el resultado de las conversaciones del cliente con el desarrollador. El cliente debería ser quien las escriba más que el desarrollador.
- Representan el inicio de la definición del cómo. No están diseñados para ser tan detallados como una especificación de diseño tradicional.
- Si una historia de usuario tiene más de 4 criterios de aceptación, debe evaluarse subdividir la historia.
- Puede añadirse un número de escenario para identificar al criterio, asociado a la historia de usuario en cuestión.

Beneficios:

- Al ser muy corta, ésta representa requisitos del modelo de negocio que pueden implementarse rápidamente (días o semanas).
- Necesitan poco mantenimiento.
- Mantienen una relación cercana con el cliente.
- Permite dividir los proyectos en pequeñas entregas.
- Permite estimar, fácilmente, el esfuerzo de desarrollo.
- Es ideal para proyectos con requisitos volátiles o no muy claros.

Limitaciones:

- Sin criterios de aceptación pueden quedar abiertas a distintas interpretaciones, haciendo difícil utilizarlas como base para un contrato.
- Se requiere un contacto permanente con el cliente durante el proyecto, lo cual puede ser difícil o costoso.
- Podría resultar difícil escalar a proyectos grandes.
- Requiere desarrolladores muy competentes.

Épicas: Se denomina épica a un conjunto de historias de usuario que se agrupan por algún denominador común. La épica representa un objetivo alcanzable que nace de la necesidad del cliente. Es un objetivo al que nos aproximamos y que esperamos alcanzar algún día. La épica no es la funcionalidad.

Características de las épicas:

- Las épicas suelen abarcar varios equipos de desarrollo.
- Recogen, normalmente, muchas historias de usuario.
- Los clientes determinan si eliminan o añaden historias dentro de cada épica.
- Una épica sirve para estructurar los temas e iniciativas (objetivos).
- Las épicas también sirven para dar flexibilidad y agilidad al proyecto.

NOTA DE CLASE 3:

Casos de Uso (CU).

Requerimientos - Tipos:

- **Funcionales:**
 - Describen una interacción entre el sistema y su ambiente. Cómo debe comportarse el sistema ante determinado estímulo.
 - Describen lo que el sistema debe hacer o, incluso, cómo NO debe comportarse.
 - Describen, con detalle, la funcionalidad del mismo.
 - Son independientes de la implementación de la solución.
 - Se pueden expresar de distintas formas.
- **No funcionales:**
 - Describen una restricción sobre el sistema que limita nuestras elecciones en la construcción de una solución al problema.
 - Del producto: Especifican el comportamiento del producto (usabilidad, eficiencia, rendimiento, espacio, fiabilidad, portabilidad).
 - Organizacionales: Se derivan de las políticas y procedimientos existentes en la organización del cliente y en la del desarrollador (entrega, implementación, estándares).
 - Externos: Interoperabilidad, legales, privacidad, seguridad, éticos.

Ingeniería de requerimientos: Es el proceso por el cual se transforman los requerimientos declarados por los clientes, ya sean hablados o escritos, a especificaciones precisas, no ambiguas, consistentes y completas del comportamiento del sistema, incluyendo funciones, interfaces, rendimiento y limitaciones.

La ingeniería de requerimientos es la disciplina para desarrollar una especificación completa, consistente y no ambigua, la cual servirá como base para acuerdos comunes entre todas las partes involucradas y en donde se describen las funciones que realizará el sistema.

También es el proceso mediante el cual se intercambian diferentes puntos de vista para recopilar y modelar lo que el sistema va a realizar. Este proceso utiliza una combinación de métodos, herramientas y actores, cuyo producto es un modelo del cual se genera un documento de requerimientos.

“Ingeniería de requerimientos” es un enfoque sistémico para recolectar, organizar y documentar los requerimientos del sistema; es también el proceso que establece y mantiene acuerdos sobre los cambios de requerimientos, entre los clientes y el equipo del proyecto.

Importancia:

- Permite gestionar las necesidades del proyecto en forma estructurada.
- Mejora la capacidad de predecir cronogramas de proyectos.
- Disminuye los costos y retrasos del proyecto.
- Mejora la calidad del software.
- Mejora la comunicación entre equipos.
- Evita rechazos de usuarios finales.

En la práctica, es un proceso iterativo donde las actividades se entrelazan.

Estudio de viabilidad: A partir de una descripción resumida del sistema, se elabora un informe que recomienda la conveniencia o no de realizar el proceso de desarrollo.

Responde a las siguientes preguntas:

- ¿El sistema contribuye a los objetivos generales de la organización? (Si no contribuye, entonces, no tiene un valor real en el negocio).
- ¿El sistema se puede implementar con la tecnología actual?
- ¿El sistema se puede implementar con las restricciones de costo y tiempo?
- ¿El sistema puede integrarse a otros que existen en la organización?

Una vez que se ha recopilado toda la información necesaria para contestar las preguntas anteriores, se debería hablar con las fuentes de información para responder nuevas preguntas y, luego, se redacta el informe, donde debería hacerse una recomendación sobre si debe continuar o no el desarrollo.

Validación de requerimientos: Es el proceso de certificar la corrección del modelo de requerimientos contra las intenciones del usuario. Trata de mostrar que los requerimientos definidos son los que estipula el sistema. Se describe el ambiente en el que debe operar el sistema. Es importante porque los errores en los requerimientos pueden conducir a grandes costos si se descubren más tarde.

- Validación: Al final del desarrollo, evaluar el software para asegurar que el software cumple los requerimientos. Hacer el software correcto.
- Verificación: El software cumple los requerimientos correctamente. Hacer el software correctamente.

Sobre estas definiciones: La validación sólo se puede hacer con la activa participación del usuario.

¿Es suficiente validar después del desarrollo del software?:

- La evidencia estadística dice que NO.
- Cuanto más tarde se detecta, más cuesta corregir (Boehm).
- Bola de nieve de defectos.
- Validar en la fase de especificación de requerimientos puede ayudar a evitar costosas correcciones después del desarrollo.

¿Contra qué se verifican los requerimientos?:

- No existen “los requerimientos de los requerimientos”.
- No puede probarse, formalmente, que un Modelo de Requerimientos es correcto. Puede alcanzarse una convicción de que la solución especificada en el modelo de requerimientos es el correcto para el usuario.

Comprenden:

- Verificaciones de validez (para todos los usuarios).
- Verificaciones de consistencia (sin contradicciones).
- Verificaciones de completitud (todos los requerimientos).
- Verificaciones de realismo (se pueden implementar).
- Verificabilidad (se puede diseñar conjunto de pruebas).

Técnicas de validación: Pueden ser manuales o automatizadas:

- Revisiones de requerimientos:
 - Informales: Los desarrolladores deben tratar los requerimientos con tantos stakeholders como sea posible.

- Formales: El equipo de desarrollo debe conducir al cliente, explicándole las implicaciones de cada requerimiento.
- Antes de una revisión formal, es conveniente realizar una revisión informal.
- Construcción de prototipos.
- Generación de casos de prueba.

Casos de Uso (CU): Proceso de modelado de las “funcionalidades” del sistema en términos de los eventos que interactúan entre los usuarios y el sistema. Tiene sus orígenes en el modelado orientado a objetos (Jacobson, 1992), pero su eficiencia en modelado de requerimientos hizo que se independice de la técnica de diseño utilizada, siendo aplicable a cualquier metodología de desarrollo. El uso de CU facilita y alienta la participación de los usuarios.

Beneficios:

- Herramienta para capturar requerimientos funcionales.
- Descompone el alcance del sistema en piezas más manejables.
- Medio de comunicación con los usuarios. Utiliza lenguaje común y fácil de entender por las partes.
- Permite estimar el alcance del proyecto y el esfuerzo a realizar.
- Define una línea base para la definición de los planes de prueba.
- Define una línea base para toda la documentación del sistema.
- Proporciona una herramienta para el seguimiento de los requisitos.

Componentes:

- Diagrama de Casos de Uso: Ilustra las interacciones entre el sistema y los actores.
- Escenarios (narración del CU): Descripción de la interacción entre el actor y el sistema para realizar la funcionalidad.

Diagrama:

- Caso de Uso: Representa un objetivo (funcionalidad) individual del sistema y describe la secuencia de actividades y de interacciones para alcanzarlo. Para que el CU sea considerado un requerimiento, debe estar acompañado de su respectivo escenario.
- Actores: Un actor inicia una actividad (CU) en el sistema. Representa un papel desempeñado por un usuario que interactúa (rol). Puede ser una persona, sistema externo o dispositivo externo que dispare un evento (sensor, reloj).
- Relaciones:
 - Asociaciones: Relación entre un actor y un CU en el que interactúan entre sí.
 - Extensiones (extends): Un CU extiende la funcionalidad de otro CU. Un CU puede tener muchos CU extensiones. Los CU extensiones sólo son iniciados por un CU.
 - Uso o inclusión (uses): Reduce la redundancia entre dos o más CU al combinar los pasos comunes de los CU.
 - Herencia: Relación entre actores donde un actor hereda las funcionalidades de uno o varios actores.

Escenarios: Se describen la interacción del escenario y los eventos alternativos.

Proceso de modelado:

- Identificar a los actores: ¿Dónde buscar actores potenciales? Documentación o manuales existentes, minutos de reunión, documentos de requerimientos. Responder a:
 - ¿Quién o qué proporciona las entradas al sistema?

- ¿Quién o qué recibe las salidas del sistema?
- ¿Se requieren interfaces con otros sistemas?
- ¿Quién mantendrá la información en el sistema?
- Identificar los CU para los requerimientos: Responder a:
 - ¿Cuáles son las principales tareas del actor?
 - ¿Qué información necesita el actor del sistema?
 - ¿Qué información proporciona el actor al sistema?
 - ¿Necesita el sistema informar al actor de eventos o cambios ocurridos?
 - ¿Necesita el actor informar al sistema de eventos o cambios ocurridos?
- Construir el diagrama.
- Realizar los escenarios.

Características importantes:

- Un CU debe representar una funcionalidad concreta.
- La descripción de los pasos en los escenarios debe contener más de un paso, para representar la interacción entre los componentes.
- El uso de condicionales en el curso normal es limitado a la invocación de excepciones, ya que este flujo representa la ejecución del caso sin alteraciones.
- Las precondiciones no deben representarse en los cursos alternativos, ya que al ser una precondición no va a ocurrir.
- Los “uses” deben ser accedidos, por lo menos, desde dos CU.

NOTA DE CLASE 4:

Diagrama de Transición de Estados (DTE).

Máquinas de estado finito: Describe al sistema como un conjunto de estados donde el sistema reacciona a ciertos eventos posibles (externos o internos). $f(S_i, C_j) = S_k$: Al estar en el estado S_i , la ocurrencia de la condición C_j hace que el sistema cambie al estado S_k .

Formalmente, un autómata finito (AF) puede ser descrito como una 5-tupla (S, Σ, T, s, A) , donde:

- Σ es un alfabeto;
- S un conjunto de estados;
- T es la función de transición;
- s es el estado inicial;
- A es un conjunto de estados de aceptación o finales.

Evento: Es un suceso significativo que debe tenerse en cuenta, que influye en el comportamiento y evolución del sistema. Tiene lugar en un punto del tiempo y carece de duración respecto a la granularidad temporal del sistema. No tiene sentido preguntarse por lo que sucede mientras está teniendo lugar el evento.

Transición: Las transiciones se producen como consecuencia de eventos. Pueden o no tener un procesamiento asociado



- Evento: obligatorio.
- Condición: opcional, depende del problema, puede haber transiciones sin condiciones.
- Acción: opcional, puede haber transiciones sin acciones.

Construcción de un DTE:

1. Identificar los estados.
2. Si hay un estado complejo se puede explotar.
3. Identificar el estado inicial: Desde el estado inicial, se identifican los cambios de estado con flechas.
4. Se analizan las condiciones y las acciones para pasar de un estado a otro
5. Se verifica la consistencia: se han definido todos los estados; se pueden alcanzar todos los estados; se puede salir de todos los estados; en cada estado, el sistema responde a todas las condiciones posibles (normales y anormales).

Ventajas del DTE:

- Mejora la comprensión: Ofrece un mapa visual que hace fácil entender cómo un sistema reacciona a diferentes eventos.
- Previene errores: Ayuda a encontrar fallos de lógica y caminos que no deberían existir, todo esto en la etapa de diseño, lo que ahorra tiempo y esfuerzo en la codificación.
- Valida el diseño: Confirma que todos los requisitos del sistema han sido considerados y que el comportamiento es coherente.

- Facilita las pruebas: Proporciona un plan para crear pruebas, asegurando que se validan todas las interacciones posibles del sistema.

Desventajas del DTE:

- Complejidad: Pueden volverse muy complejos y difíciles de manejar en sistemas con una gran cantidad de estados y transiciones
- Enfoque en un solo objeto: Un DTE, típicamente, modela el comportamiento de un único objeto o componente. Para entender el sistema completo, se necesita una colección de DTEs.
- No modela la concurrencia: Los DTEs básicos no son ideales para modelar sistemas concurrentes, donde varios estados pueden cambiar al mismo tiempo.

¿Cuándo usar DTE?:

- Sistemas de tiempo real: Cuando el software está impulsado por eventos externos, como un sistema de control para una máquina o un dispositivo.
- Modelado del comportamiento de la interfaz de usuario: Para aplicaciones con interfaces de usuario complejas, un DTE puede mostrar cómo la pantalla o un componente de la interfaz cambia en respuesta a las acciones del usuario.
- Sistemas con un número finito de estados: Cuando el componente de software puede estar en un número limitado de estados bien definidos (por ejemplo, una válvula que puede estar “abierta” o “cerrada”).
- Diseño de algoritmos de control: En el diseño de software para sistemas de control, donde el estado actual del sistema determina su próximo comportamiento.

NOTA DE CLASE 5:

Redes de Petri (RP).

Fueron inventadas por Carl Petri en la Universidad de Bonn, Alemania Occidental. Utilizadas para especificar sistemas de tiempo real en los que son necesarios representar aspectos de concurrencia. Los sistemas concurrentes se diseñan para permitir la ejecución simultánea de componentes de programación, llamadas tareas o procesos, en varios procesadores o intercalados en un solo procesador.

Las tareas concurrentes deben estar sincronizadas para permitir la comunicación entre ellas (pueden operar a distintas velocidades, deben prevenir la modificación de datos compartidos o condiciones de bloqueo). Pueden realizarse varias tareas en paralelo, pero son ejecutados en un orden impredecible. No son secuenciales.

- Los eventos se representan como transiciones (T).
- Los estados se representan como lugares o sitios (P).

Definición formal: Una estructura de Red de Petri es una 4-upla:

$C = (P, T, I, O)$, donde:

- $P = \{P_1, P_2, \dots, P_m\}$ (lugares);
- $T = \{T_1, T_2, \dots, T_m\}$ (transiciones);
- $I: T \rightarrow P$ (función de entrada);
- $O: T \rightarrow P$ (función de salida).

Multigrafo (de un nodo puede partir más de un arco), bipartito, dirigido.

Características:

- Los arcos indican, a través de una flecha, la relación entre sitios y transiciones y viceversa.
- A los lugares se les asignan tokens (fichas) que se representan mediante un número o puntos dentro del sitio. Esta asignación de tokens a lugares constituye la marcación.
- Luego de una marcación inicial, se puede simular la ejecución de la red. El número de tokens asignados a un sitio es ilimitado.
- El conjunto de tokens asociado a cada estado sirve para manejar la coordinación de eventos y estados.
- Una vez que ocurre un evento, un token puede “viajar” de uno de los estados a otro.
- Las reglas de disparo provocan que los tokens “viajen” de un lugar a otro cuando se cumplen las condiciones adecuadas.
- La ejecución es controlada por el número y distribución de los tokens.
- La ejecución de una Red de Petri se realiza disparando transiciones habilitadas.
- Una transición está habilitada cuando cada lugar de entrada tiene, al menos, tantos tokens como arcos hacia la transición.
- Disparar una transición habilitada implica remover tokens de los lugares de entrada y distribuir tokens en los lugares de salida (teniendo en cuenta la cantidad de arcos que llegan y la cantidad de arcos que salen de la transición).
- La ocurrencia de los eventos (transiciones) depende del estado del sistema.
- Una condición puede ser V (con token) o F (sin token).
- La ocurrencia de un evento está sujeta a que se den ciertas condiciones (pre) y, al ocurrir el evento, causa que se hagan verdaderas las postcondiciones.
- Las RP son asincrónicas y el orden en que ocurren los eventos es uno de los permitidos.
- La ejecución es NO DETERMINÍSTICA.

- Se acepta que el disparo de una transición es instantáneo.
- Sincronización: Para que varios procesos colaboren en la solución de un problema es necesario que compartan información y recursos, pero esto debe ser controlado para asegurar la integridad y correcta operación del sistema.
- Es importante desarrollar modelos de los sistemas de eventos discretos para estudiarlos y comprender su comportamiento.
- Existen herramientas computacionales que permiten analizar este tipo de sistemas, las cuales están basadas en análisis estadísticos y ofrecen soluciones con ciertos grados de incertidumbre.
- Por otro lado, las RP pueden ser aplicadas para la modelación de sistemas de eventos discretos, las cuales ofrecen una forma de representación gráfica y matemática de los sistemas modelados.
- La formalidad matemática de la RP proporciona herramientas de análisis para analizar los posibles estados a los que el sistema modelado pudiera alcanzar.

NOTA DE CLASE 6:

Tablas de Decisión / Análisis Estructurado.

Tablas de decisión:

Es una herramienta que permite presentar de forma concisa las reglas lógicas que hay que utilizar para decidir acciones a ejecutar en función de las condiciones y la lógica de decisión de un problema específico.

Describe el sistema como un conjunto de:

- Posibles condiciones satisfechas por el sistema en un momento dado.
- Reglas para reaccionar ante los estímulos que ocurren cuando se reúnen determinados conjuntos de condiciones.
- Acciones a ser tomadas como un resultado.

Construiremos las tablas con:

- Condiciones simples y acciones simples.
- Las condiciones toman sólo valores Verdadero o Falso.
- Hay 2^n Reglas, donde N es la cantidad de condiciones.

A partir de un enunciado, se debe:

1. Identificar las condiciones y las acciones.
 2. Completar la tabla teniendo en cuenta:
 - a) Si hay condiciones que son opuestas, debe colocarse una de ellas porque por la negativa se “obtendrá” la otra. (Si son n condiciones excluyentes, colocar n-1 en la tabla).
 - b) Las condiciones deben ser atómicas.
 3. Se construyen las reglas.
- Especificaciones completas: Aquellas que determinan acciones (una o varias) para todas las reglas posibles.
 - Especificaciones redundantes: Aquellas que marcan, para reglas que determinan las mismas condiciones, acciones iguales.
 - Especificaciones contradictorias: Aquellas que especifican, para reglas que determinan las mismas condiciones, acciones distintas.

Análisis estructurado:

- Para entender los requerimientos, se debe poder reconocer, además, cómo, se mueven los datos, los procesos o transformaciones que sufren dichos datos y sus resultados.
- La elicitación proporciona una descripción verbal del sistema. Una descripción visual puede consolidar la información.
- La técnica de análisis estructurado permite lograr una representación gráfica que permite lograr una comprensión más profunda del sistema a construir y comunicar a los usuarios lo comprendido.
- La notación no especifica aspectos físicos de implementación.
- Hace énfasis en el procesamiento o la transformación de datos conforme estos pasan por distintos procesos.

Diagrama de Flujo de Datos (DFD):

- Es una herramienta que permite visualizar un sistema como una red de procesos funcionales, conectados entre sí por “conductos” y almacenamientos de datos.
- Representa la transformación de entradas a salidas y es también llamado diagrama de burbujas.
- Es una herramienta, comúnmente, utilizada por sistemas operacionales en los cuales las funciones del sistema son de gran importancia y son más complejas que los datos que éste maneja.
- Se utiliza un rectángulo para representar una entidad externa, esto es, un elemento del sistema (por ejemplo, un elemento hardware, una persona, otro programa) u otro sistema que produce información para ser transformada por el software o recibe información producida por el software.
- Un círculo (también llamado burbuja) representa un proceso o transformación que es aplicado a los datos (o al control) y los modifica.
- Una flecha representa un flujo para uno o más elementos de datos (objetos de dato).
- Un rectángulo abierto (lado izquierdo y derecho) que representa un almacén de datos.

Se debe visualizar desde una perspectiva jerárquica de arriba hacia abajo. Pasos:

1. Redactar la lista de actividades (eventos) de la organización para determinar: entidades externas; flujos de datos; procesos; almacenes de datos.
2. Crear un diagrama de contexto que muestre las entidades externas y los flujos de datos desde y hacia el sistema.
3. Dibujar el Diagrama 0 (siguiente nivel), con procesos generales y los almacenes correspondientes.
4. Dibujar un diagrama hijo por cada uno de los procesos del Diagrama 0.

NOTA DE CLASE 7:

Modelos de Proceso.

Proceso:

- Cuando proveemos un servicio o creamos un producto, siempre seguimos una secuencia de pasos para realizar un conjunto de tareas.
- Las tareas son realizadas, usualmente, en el mismo orden.
- Se puede pensar un “conjunto ordenado de tareas” como un proceso.

¿Qué es proceso de software?: Es un conjunto de actividades y resultados asociados que producen un producto de software.

Actividades fundamentales de los procesos:

- **Especificación del software:** Consiste en el proceso de comprender y definir qué servicios se requieren del sistema, así como la identificación de las restricciones sobre la operación y desarrollo del sistema. También llamada Ingeniería de Requerimientos.
- **Desarrollo del software:** Corresponde al proceso de convertir una especificación del sistema en un sistema ejecutable. Incluye los procesos de diseño y programación. Se crea una descripción de la estructura del software que se va a implementar, los modelos y estructuras de datos, las interfaces, etc.
- **Validación del software:** Se realiza para mostrar que un sistema cumple tanto con sus especificaciones como con las expectativas del cliente. La prueba del sistema con datos de prueba simulados es una de las formas de validación. Pero también incluye inspecciones y revisiones en distintas etapas.
- **Evolución del software:** El mantenimiento es una actividad para tener en cuenta en el proceso de desarrollo de software. Eso implica también cambios y mejoras.

¿Qué es un modelo de proceso de software?:

- Es una representación simplificada de un proceso de software que presenta una visión de ese proceso.
- Estos modelos pueden incluir actividades que son partes de los procesos y productos de software, y el papel de las personas involucradas.
- Marco de referencia que contiene los procesos, las actividades y las tareas involucradas en el desarrollo, la explotación y el mantenimiento de un producto de software, abarcando la vida del sistema desde la definición de los requisitos hasta la finalización de su uso.

Características:

- Establece todas las actividades.
- Utiliza recursos, está sujeto a restricciones y genera productos intermedios y finales.
- Puede estar compuesto por subprocesos.
- Cada actividad tiene entradas y salidas definidas.
- Las actividades se organizan en una secuencia.
- Existen principios que orientan sobre las metas de cada actividad.
- Las restricciones pueden aplicarse a una actividad, recurso o producto.

Definiciones:

- **Ciclo de vida:** Proceso que implica la construcción de un producto

- Ciclo de vida del software: Describe la vida del producto de software desde su concepción hasta su implementación, entrega, utilización y mantenimiento.
- Modelos de proceso de software: Es una representación abstracta de un proceso del software.

Modelos - Tipos:

- Modelos prescriptivos: Prescriben un conjunto de elementos del proceso: actividades del marco de trabajo, acciones de la ingeniería del software, tareas, aseguramiento de la calidad y mecanismos de control. Cada modelo de proceso prescribe también un “flujo de trabajo”, es decir, de qué forma los elementos del proceso se interrelacionan entre sí.
- Modelos descriptivos: Descripción en la forma en que se realizan en la realidad.
- Ambos modelos deberían ser iguales.
- Modelos tradicionales: Formados por un conjunto de fases o actividades en las que no tienen en cuenta la naturaleza evolutiva del software. Clásico, lineal, secuencial o en cascada; modelo en V; basado en prototipos.
- Modelos evolutivos: Son modelos que se adaptan a la evolución que sufren los requisitos del sistema en función del tiempo. En espiral, evolutivo, incremental.

Modelo en Cascada:

- Las etapas se representan cayendo en cascada.
- Cada etapa de desarrollo se debe completar antes que comience la siguiente.
- Útil para diagramar lo que se necesita hacer.
- Su simplicidad hace que sea fácil explicarlo a los clientes.

Dificultades:

- No existen resultados concretos hasta que todo esté terminado.
- Las fallas más triviales se encuentran al comienzo del período de prueba y las más graves al final.
- La eliminación de fallas suele ser, extremadamente, difícil durante las últimas etapas de prueba del sistema.
- Deriva del mundo del hardware y presenta una visión de manufactura sobre el desarrollo de software.
- La necesidad de pruebas aumenta, exponencialmente, durante las etapas finales.
- “Congelar” una fase es poco realista.
- Existen errores, cambios de parecer, cambios en el ambiente.

Modelo en V:

- Demuestra cómo se relacionan las actividades de prueba con las de análisis y diseño.
- Sugiere que la prueba unitaria y de integración también sea utilizada para verificar el diseño del programa.
- La vinculación entre los lados derecho e izquierdo implica que, si se encuentran problemas durante la verificación y validación, entonces, el lado izquierdo de la V puede ser ejecutado, nuevamente, para solucionar el problema.

Modelo de prototipos: Un prototipo es un producto parcialmente desarrollado que permite que clientes y desarrolladores examinen algunos aspectos del sistema propuesto, y decidan si éste es adecuado o correcto para el producto terminado. Ésta es una alternativa de especificación para tratar mejor la incertidumbre, la ambigüedad y la volubilidad de los proyectos reales.

Tipos:

- Evolutivos: El objetivo es obtener el sistema a entregar. Permite que todo el sistema o alguna de sus partes se construyan, rápidamente, para comprender o aclarar aspectos y asegurar que el desarrollador, el usuario y el cliente tengan una comprensión unificada tanto de lo que se necesita como de lo que se propone como solución.
- Descartables: No tiene funcionalidad. Se utilizan herramientas de modelado.

Proyectos candidatos:

- Usuarios que no examinarán los modelos abstractos.
- Usuarios que no determinarán sus requerimientos inicialmente.
- Sistemas con énfasis en los formatos de E/S más que en los detalles algorítmicos.
- Sistemas en los que haya que explorar aspectos técnicos.
- Si el usuario tiene dificultad al tratar con los modelos gráficos para modelar los requerimientos y el comportamiento.
- Si se enfatiza el aspecto de la interfaz humana.

Para asegurar el éxito:

- Debe ser un sistema con el que se pueda experimentar.
- Debe ser, comparativamente, barato (< 10%).
- Debe desarrollarse rápidamente.
- Énfasis en la interfaz de usuario.
- Equipo de desarrollo reducido.
- Herramientas y lenguajes adecuados.

Modelo de desarrollo por fases: Se desarrolla el sistema de tal manera que puede ser entregado en piezas. Esto implica que existen dos sistemas funcionando en paralelo: el sistema operacional y el sistema en desarrollo.

Tipos:

- Incremental: El sistema es particionado en subsistemas de acuerdo con su funcionalidad. Cada entrega agrega un subsistema.
- Iterativo: Entrega un sistema completo desde el principio y, luego, aumenta la funcionalidad de cada subsistema con las nuevas versiones.

Modelo en espiral (Boehm):

- Combina las actividades de desarrollo con la gestión del riesgo.
- Trata de mejorar los ciclos de vida clásicos y prototipos.
- Incorpora objetivos de calidad.
- Elimina errores y alternativas no atractivas al comienzo.
- Permite iteraciones, vuelta atrás y finalizaciones rápidas.
- Cada ciclo empieza identificando: los objetivos de la porción correspondiente; las alternativas; las restricciones.
- Cada ciclo se completa con una revisión que incluye todo el ciclo anterior y el plan para el siguiente.

Cada ciclo en la espiral se divide en cuatro sectores:

1. Establecimiento de objetivos: Se identifican restricciones, se traza un plan de gestión, se identifican riesgos
2. Valoración y reducción del riesgo: Se analiza cada riesgo identificado y se determinan acciones.
3. Desarrollo y validación: Se determina el modelo de desarrollo.
4. Planeación: El proyecto se revisa y se toman decisiones para la siguiente fase.

NOTA DE CLASE 8:

Metodologías Ágiles.

Introducción:

En los años 80 y principios de los 90, existía una opinión general de que la mejor forma de obtener un mejor software era a través de una planificación cuidadosa del proyecto, la utilización de métodos de análisis y diseño, y de procesos de desarrollo de software controlados y rigurosos. En general, se realizaban sistemas críticos, desarrollados por grandes equipos, a menudo dispersos geográficamente. Sin embargo, cuando este enfoque fue aplicado a sistemas de negocio pequeños y de tamaño medio, el esfuerzo invertido era grande y, cuando cambiaban los requerimientos, se hacía esencial rehacer el trabajo. Del descontento, nacieron las metodologías ágiles.

El éxito de un desarrollo está dado por la metodología empleada, la cual nos da una dirección a seguir para su correcta conclusión. Generalmente, esta metodología lleva asociado un marcado énfasis en el control del proceso, definiendo roles, actividades, herramientas y documentación detallada. Este enfoque no resulta ser muy adecuado para proyectos actuales donde el entorno del sistema es muy cambiante y se exige una reducción de tiempo. Ante estas dificultades, muchos equipos se resignan a prescindir de las buenas prácticas, asumiendo los riesgos. En este contexto, las metodologías ágiles emergen como una posible solución.

Definición: Es un enfoque iterativo e incremental (evolutivo) de desarrollo de software.

Una Metodología Ágil es aquella en la que “se da prioridad a las tareas que dan resultados directos y que reducen la burocracia tanto como sea posible” [Fowler], adaptándose, además, rápidamente, al cambio de los proyectos. Ese enfoque ha sido utilizado desde hace más de dos décadas por un grupo de profesionales de software.

Principios:

1. Nuestra mayor prioridad es satisfacer al cliente a través de fáciles y continuas entregas de software valuable.
2. Los cambios de requerimientos son bienvenidos, aún tardíos, en el desarrollo. Los procesos Ágiles capturan los cambios para que el cliente obtenga ventajas competitivas.
3. Entregas frecuentes de software, desde un par de semanas a un par de meses, con el menor intervalo de tiempo posible entre una entrega y la siguiente.
4. Usuarios y desarrolladores deben trabajar juntos durante todo el proyecto.
5. Construir proyectos alrededor de motivaciones individuales.
6. Darles el ambiente y el soporte que ellos necesitan y confiar el trabajo dado. El diálogo cara a cara es el método más eficiente y efectivo de intercambiar información entre el equipo de desarrolladores.
7. El software que funciona es la medida clave de progreso.
8. Los procesos ágiles promueven un desarrollo sostenible. Los stakeholders, desarrolladores y usuarios deberían ser capaces de mantener un paso constante indefinidamente.
9. Atención continua a la excelencia técnica y buen diseño incrementa la agilidad.
10. Simplicidad (el arte de maximizar la cantidad de trabajo no dado) es esencial.
11. Las mejores arquitecturas, requerimientos y diseños surgen de la propia organización de los equipos.
12. A intervalos regulares, el equipo reflexiona sobre cómo volverse más efectivo, entonces, afina y ajusta su comportamiento en consecuencia.

Desventajas: En la práctica, los principios que subyacen a los métodos ágiles son, a veces, difíciles de cumplir:

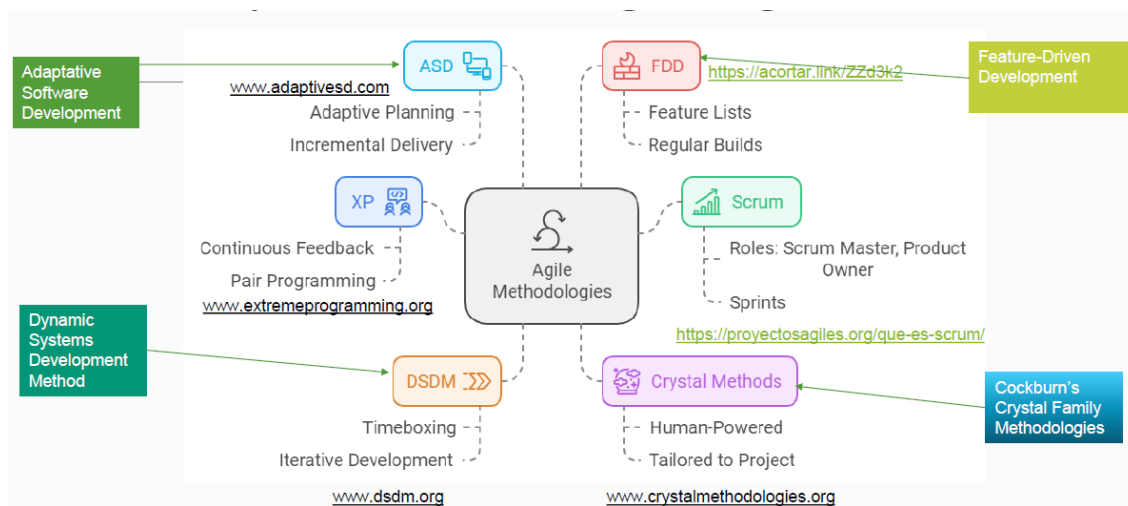
- Aunque es atractiva la idea de involucrar al cliente en el proceso de desarrollo, los representantes del cliente están sujetos a otras presiones y no intervienen, por completo, en el desarrollo del software.

- Priorizar los cambios podría ser difícil, sobre todo en sistemas donde existen muchos participantes. Cada uno, por lo general, ofrece diversas prioridades a diferentes cambios.
- Mantener la simplicidad requiere trabajo adicional. Bajo la presión de fechas de entrega, es posible que los miembros del equipo carezcan de tiempo para realizar las simplificaciones deseables al sistema.
- Muchas organizaciones, especialmente las grandes compañías, pasan años cambiando su cultura, de tal modo que los procesos se definan y continúen. Para ellas, resulta difícil moverse hacia un modelo de trabajo donde los procesos sean informales y estén definidos por equipos de desarrollo.

Por lo general, el documento de requerimientos del software forma parte del contrato entre el cliente y el proveedor. Como en los métodos ágiles se minimiza la documentación, suele ser complejo reglamentarlo.

La mayoría de los libros que describen los métodos ágiles y las experiencias con estos hablan del uso de dichos métodos para el desarrollo de nuevos sistemas. Sin embargo, una enorme cantidad de esfuerzo en ingeniería de software se usa en el mantenimiento y la evolución de los sistemas de software existentes. Al no existir documentación, se complejizaría.

Principales Metodologías Ágiles:



eXtreme Programming: Es una disciplina de desarrollo de software basado en los valores de la sencillez, la comunicación, la retroalimentación, la valentía y el respeto. Su acción consiste en llevar a todo el equipo reunido en la presencia de prácticas simples, con suficiente información para ver dónde están y para ajustar las prácticas a su situación particular.

Características:

- Desarrollo iterativo e incremental: pequeñas mejoras, unas tras otras.
- Pruebas unitarias continuas, frecuentemente repetidas y automatizadas, incluyendo pruebas de regresión.
- Programación en parejas.
- Frecuente integración del equipo de programación con el cliente o usuario.
- Corrección de todos los errores antes de añadir nueva funcionalidad.
- Refactorización del código.
- Propiedad del código compartida.
- Simplicidad en el código.

Principios básicos: comunicación, coraje, respeto, retroalimentación, simplicidad.

Características esenciales: historias de usuario, roles, proceso, prácticas.

Roles:

- Programador (Programmer):
 - Responsable de decisiones técnicas.
 - Responsable de construir el sistema.
 - Sin distinción entre analistas, diseñadores o codificadores.
 - En XP, los programadores diseñan, programan y realizan las pruebas.
- Jefe de Proyecto (Manager):
 - Organiza y guía las reuniones.
 - Asegura condiciones adecuadas para el proyecto.
- Cliente (Customer):
 - Es parte del equipo.
 - Determina qué construir y cuándo.
 - Establece las pruebas funcionales.
- Entrenador (Coach):
 - Responsable del proceso.
 - Tiende a estar en un segundo plano a medida que el equipo madura.
- Encargado de Pruebas (Tester):
 - Ayuda al cliente con las pruebas funcionales.
 - Se asegura de que las pruebas funcionales se superan.
- Rastreador (Tracker):
 - Metric Man.
 - Observa sin molestar.
 - Conserva datos históricos.

Proceso: El ciclo de vida consiste en:

1. Exploración:
 - Los clientes plantean las historias de usuario que son de interés para la primera entrega del producto.
 - El equipo de desarrollo se familiariza con las herramientas, tecnologías y prácticas que se utilizarán en el proyecto.
 - Se construye un prototipo.
 - La fase de exploración toma de pocas semanas a pocos meses, dependiendo del tamaño y familiaridad que tengan los programadores con la tecnología.
2. Planificación:
 - El cliente establece la prioridad de cada historia de usuario.
 - Los programadores realizan una estimación del esfuerzo.
 - Se toman acuerdos sobre el contenido de la primera entrega y se determina un cronograma en conjunto con el cliente.
 - Esta fase dura unos pocos días.
3. Iteraciones:
 - El Plan de Entrega está compuesto por iteraciones de no más de tres semanas.
 - El cliente es quien decide qué historias se implementarán en cada iteración.
 - Al final de la última iteración, el sistema estará listo para entrar en producción.
 - Esta fase incluye varias iteraciones sobre el sistema antes de ser entregado.
4. Producción:

- Esta fase requiere de pruebas adicionales y revisiones de rendimiento antes de que el sistema sea trasladado al entorno del cliente.
 - Al mismo tiempo, se deben tomar decisiones sobre la inclusión de nuevas características a la versión actual, debido a cambios durante esta fase.
5. Mantenimiento:
- Mientras la primera versión se encuentra en producción, el proyecto XP debe mantener el sistema en funcionamiento al mismo tiempo que desarrolla nuevas iteraciones.
 - La fase de mantenimiento puede requerir nuevo personal dentro del equipo y cambios en su estructura.
6. Muerte:
- Es cuando el cliente no tiene más historias para ser incluidas en el sistema.
 - Se genera la documentación final del sistema y no se realizan más cambios en la arquitectura.
 - La muerte del proyecto también ocurre cuando el sistema no genera los beneficios esperados por el cliente o cuando no hay presupuesto para mantenerlo.



Prácticas:

1. Testing: Los programadores, continuamente, escriben pruebas unitarias, las cuales deben correr sin problemas para que el desarrollo continúe. Los clientes escriben pruebas demostrando que las funcionalidades están terminadas.
2. Refactoring: Actividad constante de reestructuración del código con el objetivo de remover duplicación de código, mejorar su legibilidad, simplificarlo y hacerlo más flexible para facilitar los posteriores cambios.
3. Programación de a pares: Todo el código de producción es escrito por dos programadores en una máquina.
4. Propiedad colectiva del código: Cualquiera puede cambiar código en cualquier parte del sistema en cualquier momento. Motiva a contribuir con nuevas ideas, evitando, a la vez, que algún programador sea imprescindible.
5. Integración continua: Cada pieza de código es integrada en el sistema una vez que esté lista. Así, el sistema puede llegar a ser integrado y construido varias veces en un mismo día. Reduce la fragmentación de los esfuerzos de los desarrolladores por falta de comunicación sobre lo que puede ser reutilizado o compartido.
6. Semana de 40 horas: Se debe trabajar un máximo de 40 horas por semana. El trabajo extra desmotiva al equipo. Los proyectos que requieren trabajo extra para intentar cumplir con los plazos suelen al final ser entregados con retraso. En lugar de esto, se puede realizar el juego de la planificación para cambiar el ámbito del proyecto o la fecha de entrega.
7. Cliente en el lugar de desarrollo: El cliente tiene que estar presente y disponible todo el tiempo para el equipo.
8. Estándares de codificación: Los programadores escriben todo el código de acuerdo con reglas que enfatizan la comunicación a través del mismo.

Scrum: Es un marco de trabajo utilizado para desarrollar productos complejos donde se aplican, de manera regular, un conjunto de buenas prácticas para trabajar colaborativamente y obtener el mejor resultado posible de un proyecto. Estas prácticas se apoyan unas a otras y su selección tiene origen en un estudio de la manera de trabajar de equipos, altamente, productivos. Scrum se define como “una manera simple de manejar problemas

complejos”, proporcionando un paradigma de trabajo que soporta la innovación y permite que equipos auto-organizados entreguen resultados de alta calidad en tiempos cortos. En Scrum, se realizan entregas parciales y regulares del resultado final del proyecto, priorizadas por el beneficio que aportan al receptor del proyecto.

Principios:

- Eliminar el desperdicio: No generar artefactos ni perder el tiempo haciendo cosas que no le suman valor al cliente.
- Construir la calidad con el producto: La idea es inyectar la calidad, directamente, en el código desde el inicio.
- Crear conocimiento: En la práctica, no se puede tener el conocimiento antes de empezar el desarrollo.
- Diferir las decisiones: Tomar las decisiones en el momento adecuado, esperar hasta ese momento, ya que uno tiene más información a medida que va pasando el tiempo. Si se puede esperar, mejor.
- Entregar rápido: Debe ser una de las ventajas competitivas más importantes.
- Respetar a las personas: La gente trabaja mejor cuando se encuentra en un ambiente que la motive y se sienta respetada.
- Optimizar el todo: Optimizar todo el proceso, ya que el proceso es una unidad, y, para lograr tener éxito y avanzar, hay que tratarlo como tal.

Roles:

- Product Owner (Propietario): Conoce y marca las prioridades del proyecto o producto.
- Scrum Master (Jefe): Es la persona que asegura el seguimiento de la metodología guiando las reuniones y ayudando al equipo ante cualquier problema que pueda aparecer. Su responsabilidad es, entre otras, la de hacer de paraguas ante las presiones externas.
- Scrum Team (Equipo): Son las personas responsables de implementar la funcionalidad o funcionalidades elegidas por el Product Owner.
- Usuarios o Cliente: Son los beneficiarios finales del producto, y son quienes, viendo los progresos, pueden aportar ideas, sugerencias o necesidades.

Artefactos:

- Product Backlog: Es la lista maestra que contiene toda la funcionalidad deseada en el producto. La característica más importante es que la funcionalidad se encuentra ordenada por un orden de prioridad.
- Sprint Backlog: Es la lista que contiene toda la funcionalidad que el equipo se comprometió a desarrollar durante un Sprint determinado.
- Burndown Chart: Muestra un acumulativo del trabajo hecho, día a día.
- Entre otros.

Proceso: Scrum es iterativo e incremental. Se busca poder atacar todos los problemas que surgen durante el desarrollo del proyecto. El nombre Scrum se debe a que, durante los Sprints, lo que serían las fases de desarrollo se solapan, de manera que no es un proceso de cascada por cada iteración, sino que tenemos todas estas etapas juntas que se ejecutan una y otra vez, hasta que se crea suficiente.

¿Cuándo usar Scrum?: Scrum está pensado para ser aplicado en proyectos en donde el “caos” es una constante, aquellos proyectos en los que tenemos requerimientos dinámicos y que tenemos que implementar tecnología de punta. Esos proyectos difíciles, que, con los enfoques tradicionales, se hace imposible llegar a buen puerto.

KANVAN: Es un enfoque Lean de desarrollo de software ágil.

Literalmente, Kanban es una palabra japonesa que significa “tarjeta visual”. En Toyota, Kanban es el término que se utiliza para el sistema de señalización visual y física que une todo el sistema de producción Lean.

Kanban se presenta como una herramienta de gestión que prescribe sólo 3 prácticas: limitar el trabajo en curso; visualizar el flujo de trabajo; medir el tiempo promedio de entrega.

Esto la convierte en una herramienta simple, pero, a la vez, muy potente.

Esta metodología ha ganado popularidad en el mantenimiento de multiproyectos de diferentes características.

Kanban trata de administrar el flujo de trabajo. Inicialmente, no reemplaza nada que la organización haga, simplemente impulsa el cambio.

A diferencia de otras metodologías, no define ningún rol ni prescribe una secuencia de pasos definidos para llevar a cabo el desarrollo del proyecto. Su implementación es sencilla (3 prácticas).

Es habitual combinar en las prácticas de otras metodologías ágiles los tableros Kanban, que son la herramienta ágil por excelencia de esta metodología, donde se visualizan las tareas a realizar, las realizadas y lo pendiente. Estos son una herramienta para el control visual del sistema que ayuda a un equipo a visualizar, explícitamente, el proceso de desarrollo.

A primera vista, el tablero Kanban se asemeja a un proceso en cascada. Pero, en la práctica, evita, explícitamente, los problemas de un proceso de estas características mediante la aplicación de lotes pequeños de trabajo y el desarrollo incremental.

Está compuesto por una serie de columnas que representan los diversos estados que atraviesa un requerimiento durante el proceso de desarrollo. Las tarjetas se mueven de un estado a otro mostrando la evolución hasta que haya sido aceptado por el cliente.

Cada columna tendrá un límite para el trabajo en curso. Cuando una tarea es completada en una columna, ésta se mueve a la siguiente. De esta forma, se crea un espacio libre en la columna actual representando capacidad de trabajo disponible. El equipo, entonces, toma una tarea terminada desde el estado anterior y la desplaza hacia el estado actual, ya que hay capacidad para procesarla.

NOTA DE CLASE 9:

Desarrollo de Software Basado en Modelos / Calidad.

Desarrollo de software basado en modelos:

Hacia fines de los 70', De Marco introdujo el concepto de desarrollo de software basado en modelos. Destacó que la construcción de un sistema de software debe ser precedida por la construcción de un modelo, tal como se realiza en otros sistemas ingenieriles.

Un modelo del sistema consiste en una conceptualización del dominio del problema y actúa como una especificación precisa de los requerimientos que el sistema de software debe satisfacer. (Abstracción de elementos del problema, comunicación, negociación con el usuario).

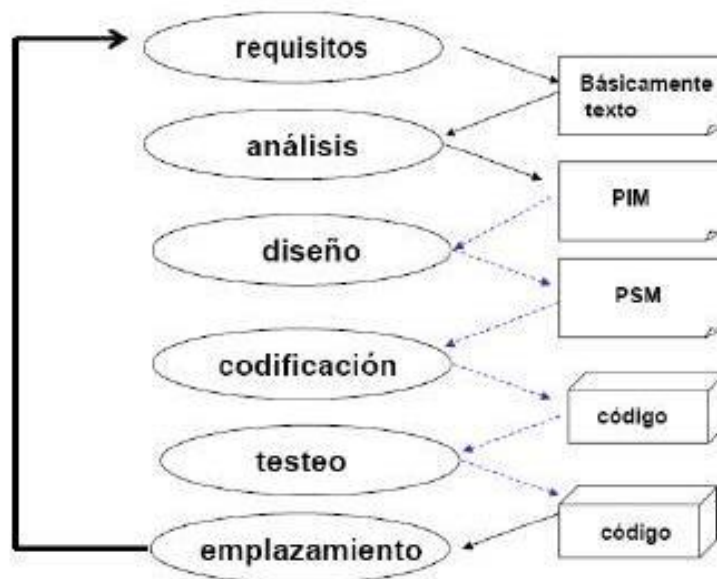
El adjetivo «dirigido» en MDD, a diferencia de «basado», enfatiza que este paradigma asigna a los modelos un rol central y activo: son, al menos, tan importantes como el código fuente.

Model Driven Development (MDD) promueve enfatizar los siguientes puntos claves:

- Mayor nivel de abstracción en la especificación tanto del problema a resolver como de la solución correspondiente.
- Aumento de confianza en la automatización asistida por computadora para soportar el análisis, el diseño y la ejecución.
- Uso de estándares industriales como medio para facilitar las comunicaciones, la interacción entre diferentes aplicaciones y productos, y la especialización tecnológica.
- Los modelos son los conductores primarios en todos los aspectos del desarrollo de software.

Los modelos pasan de ser entidades contemplativas (es decir, artefactos que son interpretados por los diseñadores y programadores) para convertirse en entidades productivas a partir de las cuales se deriva la implementación en forma automática.

Ciclo de vida del software dirigido por modelos:



Modelos de MDD:

- Platform Independent Model (PIM): Un modelo de un sistema que no contiene información acerca de la plataforma o la tecnología que es usada para implementarlo.
- Platform Specific Model (PSM): Un modelo de un sistema que incluye información acerca de la tecnología específica que se usará para su implementación sobre una plataforma específica.
- Transformación de modelos: Especifica el proceso de conversión de un modelo en otro modelo del mismo sistema.

Cada transformación incluye, al menos, un PIM, un Modelo de la Plataforma, una Transformación y un PSM.

¿Qué es una transformación?: En general, se puede decir que una definición de transformación consiste en una colección de reglas, las cuales son especificaciones no ambiguas de las formas en que un modelo (o parte de él) puede ser usado para crear otro modelo (o parte de él). El patrón MDD es, normalmente, utilizado sucesivas veces para producir una sucesión de transformaciones.

Orígenes de MDD: MDD es la evolución natural de la ingeniería de software basada en modelos enriquecida mediante el agregado de transformaciones automáticas entre modelos. Si bien MDD define un nuevo paradigma para el desarrollo de software, sus principios fundamentales no constituyen, realmente, nuevas ideas, sino que son reformulaciones y asociaciones de ideas anteriores. La técnica de transformación se asemeja al proceso de abstracción y refinamiento presentado por Dijkstra.

Beneficios de MDD:

- Incremento en la productividad (modelos y transformaciones).
- Adaptación a los cambios tecnológicos.
- Adaptación a los cambios de requisitos.
- Consistencia (automatización).
- Re-uso (de modelos y transformaciones).
- Mejoras en la comunicación con los usuarios y la comunicación entre los desarrolladores (los modelos permanecen actualizados).
- Captura de la experiencia (cambio de experto).
- Los modelos son productos de larga duración (resisten cambios).
- Posibilidad de demorar decisiones tecnológicas.

Calidad:

Definición: Propiedad o conjunto de propiedades inherentes a algo, que permiten juzgar su valor.

Calidad es un concepto manejado con bastante frecuencia en la actualidad, pero, a su vez, su significado es percibido de distintas maneras.

Al hablar de bienes y/o servicios de calidad, la gente se refiere, normalmente, a bienes de lujo o excelentes, con precios elevados.

Su significado sigue siendo ambiguo y muchas veces su uso depende de lo que cada uno entiende por calidad, por lo cual es importante comenzar a unificar su definición.

¿Qué es la calidad?:

- Un producto de calidad es un producto de lujo.
- La calidad es intangible y, por lo tanto, no mensurable.

- Los problemas son originados por los trabajadores de producción.
- La calidad se origina en el departamento de calidad.

La calidad es relativa a las personas, a su edad, a las circunstancias de trabajo, el tiempo. El tiempo varía las percepciones.

A lo largo de la historia, se han desarrollado filosofías o culturas de calidad, de las cuales algunas han sobresalido porque han tenido resultados satisfactorios. A los que realizaron estas filosofías se los ha llamado Maestros o Gurús de la Calidad.

Las principales normas internacionales definen la calidad como:

- “El grado en el que un conjunto de características inherentes cumple con los requisitos” (ISO 9000).
- “Conjunto de propiedades o características de un producto o servicio que le confieren aptitud para satisfacer unas necesidades expresadas o implícitas” (ISO 8402).

Sistemas de información: Es el conjunto de personas, datos, procesos y tecnología de información que interactúan para recopilar, procesar, guardar y proporcionar como salida la información necesaria para brindar soporte a una organización. Un sistema de información abarca más que el aspecto, meramente, computacional, pues no sólo hemos de tener en cuenta estas herramientas, sino también el modo de organizar dichas herramientas y de obtener la información necesaria para el correcto funcionamiento de la empresa.

Calidad de los sistemas de información: La importancia de los sistemas de información (SI) en la actualidad hace necesario que las empresas de tecnología hagan mucho hincapié en los estándares (o normas) de calidad. Stylianou y Kumar plantean que se debe apreciar la calidad desde un todo, donde cada parte que la componen debe tener su análisis de calidad.

Calidad de software: La calidad del software se ha mejorado, significativamente, en estos últimos años, en particular por una mayor conciencia de la importancia de la gestión de la calidad y la adopción de técnicas de gestión de la calidad para desarrollo en la industria del software. Se divide en:

- Calidad del producto obtenido: La estandarización del producto define las propiedades que debe satisfacer el producto software resultante.
- Calidad del proceso de desarrollo: La estandarización del proceso define la manera de desarrollar el producto software.

Sin un buen proceso de desarrollo, es casi imposible obtener un buen producto.

Norma/Modelo de calidad SQuaRE ISO/IEC 25010 - Características:

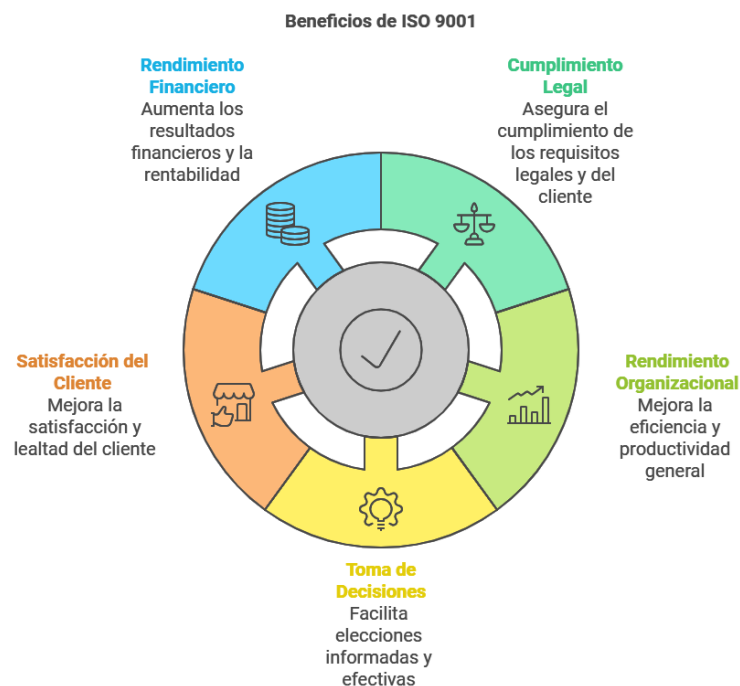
- Portabilidad.
- Seguridad.
- Facilidad de mantenimiento.
- Compatibilidad.
- Funcionalidad.
- Confiabilidad.
- Facilidad de uso.
- Eficiencia.

CMM (1993) - CMMI (2000):

- Es un modelo de evaluación de los procesos de una organización (Capability Maturity Model).
- Fue desarrollado, inicialmente, para los procesos de desarrollo e implementación de software por la Universidad Carnegie-Mellon para el Software Engineering Institute (SEI).
- Marco de referencia para desarrollar procesos efectivos.
- Proporciona un marco estructurado para evaluar los procesos actuales de la organización, establecer prioridades de mejora e implementar esas mejoras.
- En diciembre de 2000, el SEI publicó un nuevo modelo, el CMMI o “Modelo de Capacidad y Madurez - Integración”, con el objetivo de integrar distintos modelos.
- CMMI posee dos vistas que permiten un enfoque diferente según las necesidades de quien vaya a implementarlo:
 - Escalonado: Centra su foco en la madurez de la organización. Igual que CMM.
 - Continuo: Enfoca las actividades de mejora y evaluación en la capacidad de los diferentes procesos. Presenta 6 (seis) niveles de capacidad. Los niveles de capacidad indican qué tan bien se desempeña la organización en un área de proceso individual.

ISO (International Organization for Standardization): La familia ISO 9000 es un conjunto de normas de “gestión de la calidad” aplicables a cualquier tipo de organización, con el objetivo de obtener mejoras en la organización y, eventualmente, arribar a una certificación, punto importante a la hora de competir en los mercados globales.

- ISO - 9001:2015 -Quality management system-Requirements: Norma publicada por ISO en el año 2015.
- IRAM - ISO 9001:2015 -Sistema de gestión de la calidad-Requisitos: Norma publicada por ISO y traducida por IRAM.
- ISO 90003:2004: Basada ISO 9001:2000. Directrices para la interpretación en el proceso de software. Proporciona una guía para identificar las evidencias dentro del proceso de software para satisfacer los requisitos de la ISO 9001.
- SGC - Mejora continua: Implica un compromiso constante por identificar oportunidades de mejora, implementar cambios y evaluar su impacto. En el contexto de la Ingeniería de Software, la mejora continua busca optimizar los procesos de desarrollo, aumentar la calidad del software y satisfacer mejor las necesidades de los clientes. Ciclo PDCA (Plan-Do-Check-Act): Un ciclo iterativo para planificar, implementar, verificar y actuar sobre las mejoras.



Resumiendo:

Calidad de producto de software	Se evalúa la calidad mediante	ISO/IEC 25000	Está compuesto por distintos modelos. Define características que pueden estar presentes o no en el producto. La norma nos permite evaluar si están presentes o no , y de qué manera evaluarlas. Ej: Seguridad, Compatibilidad, Seguridad. Etc.
Calidad de proceso de desarrollo de software	Se evalúa la calidad mediante	ISO/IEC 12207	ISO/IEC 12207 establece un modelo de procesos para el ciclo de vida del software. Define cómo debería ser el modelo de proceso para ser completo y con calidad. Actividades, tareas etc.
		ISO/IEC 15504 (reemplazada por ISO 33000)	Es una norma internacional para establecer y mejorar la capacidad y madurez de los procesos. Define que se debe tener en cuenta para evaluar el modelo de proceso y concluir si es completo y con calidad.
		ISO/IEC 90003	Proporciona una guía sobre cómo aplicar la ISO 9001 en procesos de software
		CMMI	Proporciona un marco estructurado para evaluar los procesos actuales de la organización, establecer prioridades de mejora, e implementar esas mejoras. Se utiliza para organizaciones desarrolladoras de software de medianas a grandes dimensiones
Calidad de Procesos/Servicios en general	se evalúa mediante	ISO 9001	La Norma ISO 9001 determina los requisitos para establecer un Sistema de Gestión de la Calidad. Forma parte de la familia ISO 9000, que es un conjunto de normas de "gestión de la calidad" aplicables a cualquier tipo de organización con el objetivo de obtener mejoras en la organización y, eventualmente arribar a una certificación, punto importante a la hora de competir en los mercados globales.